Introduction to Computer Architecture
Chapter 3

Arithmetic for Computers

Hyungmin Cho

Department of Computer Science and Engineering
Sungkyunkwan University

FLOATING POINT NUMBERS

Floating Point (1)

- What would be the output?

```
double a, b, c, d;
a = 0.1;
b = 0.7;
c = 0.07;

d = a*b;

if(c==d) printf("%f is equal to %f\n",c,d);
else     printf("%f is NOT equal to %f\n",c,d);
```

```
0.070000 is NOT equal to 0.070000
```

Floating Point (2)

- Let's see how an integer number is represented

```
int i1 = 58;

printf("i1 = %d\n", i1);
printf("i1 = 0x%X\n", i1);
```

decimal

```
i1 = 58
i1 = 0x3A
```

0x0000003A

Floating Point (3)

- Let's see how a non-integer number is represented

```
float f1 = 58.0;

printf("f1 = %f\n", f1);
printf("f1 = 0x%X\n", f1);
```

58 ⇒ 111010,
e=5 + 127 = 132 → 1000 0100

```
warning: format '%X' expects argument of type 'long unsigned int',
but argument 2 has type 'double'
```

0 100|0010|0111|0
0X 4268 0000

```
f1 = 58.000000
f1 =
```

Floating Point (4)

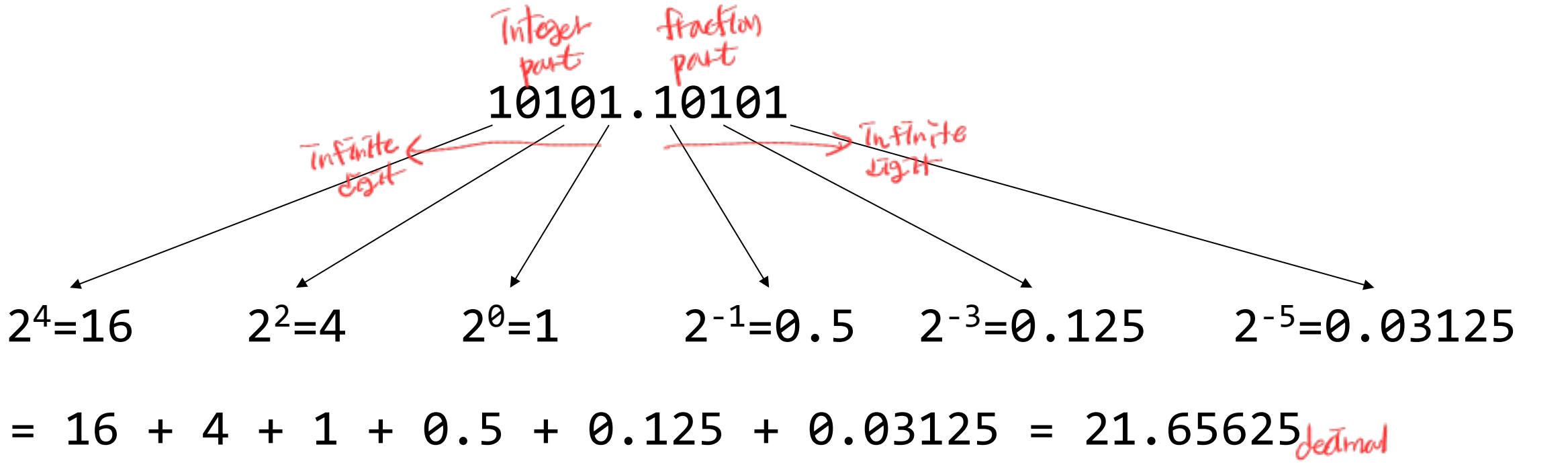
- Let’s see how a non-integer number is represented

```
float f1 = 58.0;
unsigned int u1 = *((unsigned int *) &f1);

printf("f1 = %f\n", f1);
printf("u1 = 0x%X\n", u1);
```

```
f1 = 58.000000
u1 = 0x42680000
```

Representing Non-Integer Numbers in Binary



- Question: How can we 'encode' the binary representation in computers?
  - Fixed-point format?
    - Fixed number of bits for integer / fractional parts
    - Limited range!

$2^{15}-1$ 까지만 표현가능

Floating Point

- Scientific notation (Significand + Magnitude)
  - -2.34×10^{56} $= -234 \times 10^{54}$ $= -0.234 \times 10^{57}$
  - $+0.002 \times 10^{-4}$ $= +0.2 \times 10^{-6}$ $= +2.0 \times 10^{-7}$
  - $+987.02 \times 10^{9}$ $= +9.8702 \times 10^{11}$ $= +98702 \times 10^{7}$

significand (or *coefficient*)

exponent

- Normalized (or *standard form*)
  - $1 \le |\text{significand}| < 10$

Binary Representation

$$10101.10101$$

$$= 10101.10101 \times 2^0$$
$$= 1.0101101010 \times 2^4$$

Range of the significand in binary normalized format:

$1 \le |\text{significand}| < 2$

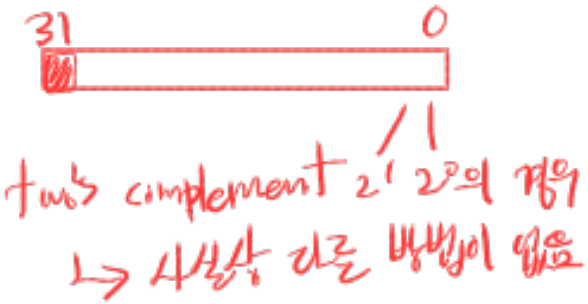
Binary number normalized format:

$\pm 1.xxxxxxx_2 \times 2^{yyyy}$

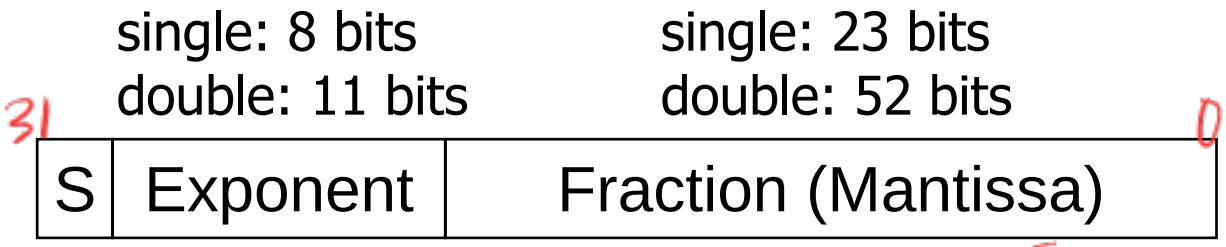
① sign bit ② mantissa ③ exponent

Floating Point Standard

- Defined by IEEE Std 754-1985
- Developed in response to divergence of representations
  - Portability issues for scientific code
- Now almost universally adopted
- Single precision (32-bit) → float type in C
- Double precision (64-bit) → double type in C



IEEE Floating-Point Format

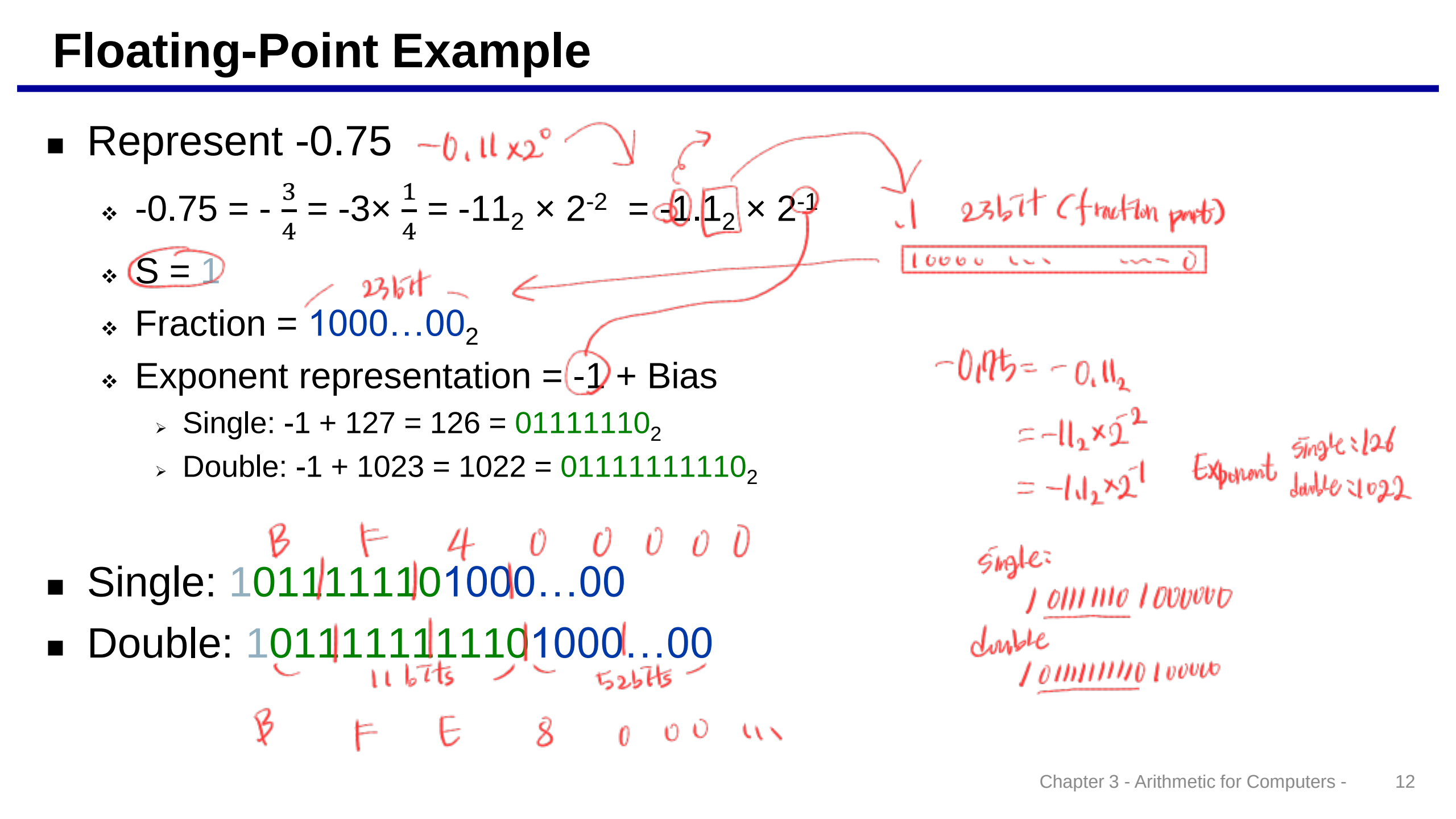


$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- S: **sign** bit (0 ⇒ non-negative, 1 ⇒ negative)
- Normalize significand: 1.0 ≤ |significand| < 2.0
  - Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
  - Significand is **Fraction** with the "1." restored
- **Exponent**: excess representation: actual exponent + Bias
  - Ensures exponent is unsigned
  - Single: Bias = 127; Double: Bias = 1023

Floating-Point Example

- Represent -0.75
  - $-0.75 = -\frac{3}{4} = -3 \times \frac{1}{4} = -11_2 \times 2^{-2} = -1.1_2 \times 2^{-1}$
  - S = 1
  - Fraction = $1000\ldots00_2$
  - Exponent representation = -1 + Bias
    - Single: -1 + 127 = 126 = 01111110_2
    - Double: -1 + 1023 = 1022 = 01111111110_2
- Single: $1011\,1111\,1010\,0000\ldots00$
- Double: $1011\,1111\,1110\,1000\ldots00$



Floating-Point Example

- What number is represented by the single-precision float
  1 10000001 01000…00
  - S = 1
  - Fraction = $01000…00_2$
  - Exponent representation = 10000001_2 = 129

- $x = (-1)^1 \times (1 + 01_2) \times 2^{(129 - 127)}$
  $= (-1) \times 1.25 \times 2^2$
  $= -5.0$

Single-Precision Range

- Exponent representations “00000000” and “11111111” are reserved

- Smallest value
  - Exponent representation: 00000001 ⇒ actual exponent = 1 - 127 = -126
  - Fraction: 000…00 ⇒ significand = 1.0
  - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$

- Largest value
  - Exponent representation: 11111110 ⇒ actual exponent = 254 - 127 = +127
  - Fraction: 111…11 ⇒ significand ≈ 2.0
  - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Floating Point (5)

- What would be the output?

```
float f2 = -0.1;
unsigned int u2 = *((unsigned int *) &f2);

printf("f2 = %f\n", f2);
printf("f2 = %3.20f\n", f2);
printf("u2 = 0x%X\n", u2);
```

Integer보다 범위는 넓지만 precision이 한계가 있다

```
f2 = -0.100000
f2 = -0.10000000149011611938
u2 = 0xBDCCCCCD
```

20

Limited Precision

- Let's represent 0.1 in single-precision Floating Point
  - $0.1 = \frac{1}{16} + \frac{1}{32} + \frac{1}{256} + \frac{1}{512} + \frac{1}{4096} \ldots\ldots$ (2^{-4} 2^{-5} 2^{-8} 2^{-9})
  - $\sum \frac{1}{2^n} = 0.000110011\ldots$
    $= 1.10011\sim \times 2^{-4}$
  - Mantissa value는 original significand의 값이 다를 수 있음
  - Exponent = -4 → -4+127 = 123 = 01111011_2
  - Significand = 1.10011001100110011001100110011001100110011001100… (계속 반복)
  - Mantissa = 10011001100110011001101 (23 bits, round up)
  - sign exponent mantissa
  - 0 01111011 10011001100110011001101
  - 0011 1101 1100 1100 1100 1100 1100 1101 → 0x3DCCCCCD
  - 0x3DCCCCCD → 0.100000001490116119384765625
    - 0011 → 1011
    - 0.1에 비해 0x3DCCCCCD (다른 slide)
    - 0.1보다는 꽤 커짐
    - round에 따라 값이 달라짐
    - round down 하면 0.1보다 작아짐

Floating-Point Addition

- Consider a 4-digit decimal example
  - $9.999 \times 10^{1} + 1.610 \times 10^{-1}$

1. Align decimal points
   - Shift number with smaller exponent
   - $9.999 \times 10^{1} + 0.016 \times 10^{1}$
2. Add significands
   - $9.999 \times 10^{1} + 0.016 \times 10^{1} = 10.015 \times 10^{1}$
3. Normalize result & check for over/underflow
   - 1.0015×10^{2}
4. Round and renormalize if necessary
   - 1.002×10^{2}

Chapter 3 — Arithmetic for Computers — 17

Floating-Point Addition

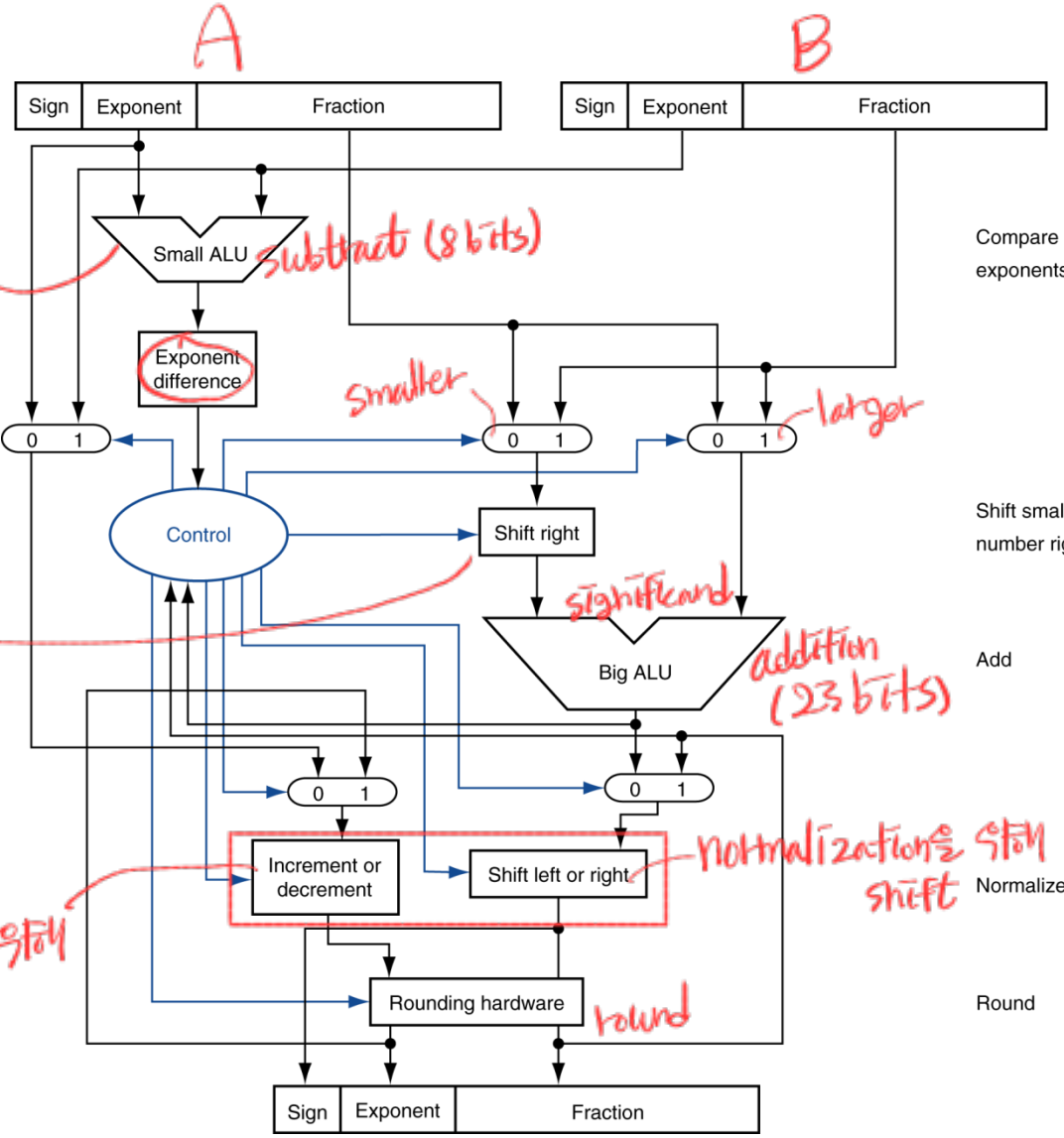
- Now consider a 4-digit binary example
  - $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ (0.5 + –0.4375)

1. Align binary points
   - Shift number with smaller exponent
   - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$
2. Add significands
   - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$
3. Normalize result & check for over/underflow
   - $1.000_2 \times 2^{-4}$, with no over/underflow
4. Round and renormalize if necessary
   - $1.000_2 \times 2^{-4}$ (no change) = 0.0625

FP Adder Hardware

- Much more complex than an integer adder
- Doing it in one clock cycle would take too long
  - Much longer than integer operations
- FP adder usually takes several cycles
  - Can be pipelined

FP Adder Hardware 간략한 설명



보통 기계는 floating point 연산을 가능하지 않음 (연산이 복잡하기 때문)

↳ CPU가 integer operation만 가능하고 Emulator를 통해 floating point operation을 소프트웨어로 구현하는 것

↳ 그래서 너무 느림, 대부분의 하드웨어적으로 가능하는 게 좋음.

floating point는 기본적으로 integer보다 더 많은 경우 사용

등이 보편적인 CPU에서 많이 사용

어느 exponent가 더 큰지 확인

Subtract (8 bits)

smaller

larger

exponent가 더 작은 값을 큰 값에 맞춤

e.g) 1.0×2^{-2} , 1.0×2^{-3}

↓

0.1×2^{-2}

right shift

significand

addition (23 bits)

normalization을 위해 shift

normalization을 위해 exponent 조정

round

Standard format이면 normalization 생략

Why use Bias in Exponent?

- Consider two numbers: 2.5 and 0.75
  - $2.5 = 10.1_2 = 1.01_2 \times 2^1$
  - $0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$

- If no bias is used (2's complement representation for exponent),
  - 2.5
    - Mantissa: $01000\ldots00_2$ Exponent representation: 1 = 00000001_2
    - Binary representation: $0000000001000\ldots00_2$ = 0x00A00000
  - 0.75
    - Mantissa: $10000\ldots00_2$ Exponent representation : -1 = 11111111_2
    - Binary representation: $0111111110000\ldots00_2$ = 0x7FC00000

Which one is bigger?

Why use Bias in Exponent?

- Consider two numbers: 2.5 and 0.75
  - $2.5 = 10.1_2 = 1.01_2 \times 2^1$
  - $0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$

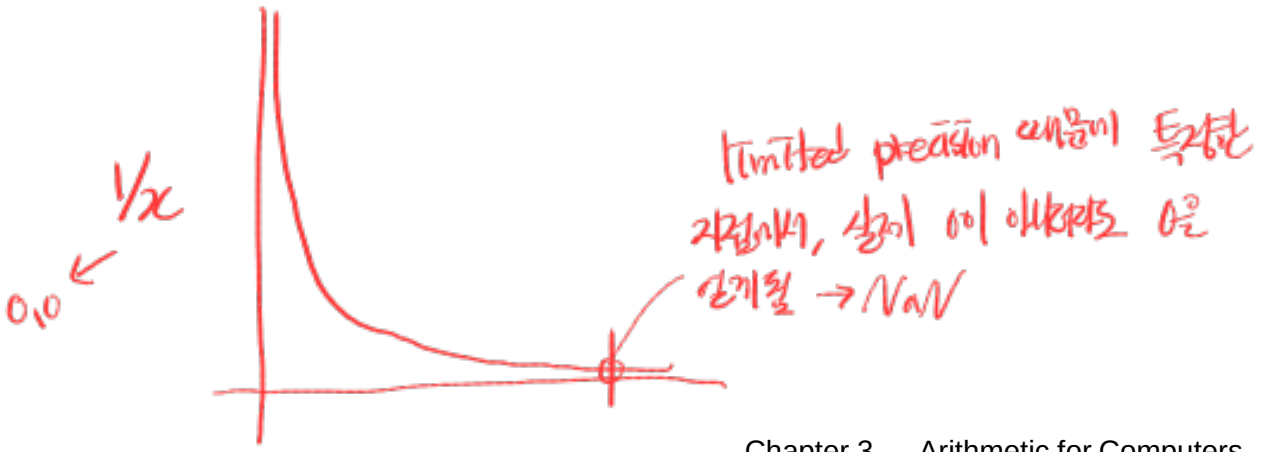
floating point의 크기를 비교할 때는 Integer처럼 비교 및 판단 가능

- If bias (127) is used,
  - 2.5
    - Mantissa: $01000\ldots00_2$ Exponent representation : 1 + 127 = 10000000_2
    - Binary representation: $01000000001000\ldots00_2$ = 0x40200000
  - 0.75
    - Mantissa: $10000\ldots00_2$ Exponent representation : -1 + 127 = 01111110_2
    - Binary representation: $00111111010000\ldots00_2$ = 0x3F400000

Which one is bigger?

Infinities and NaNs

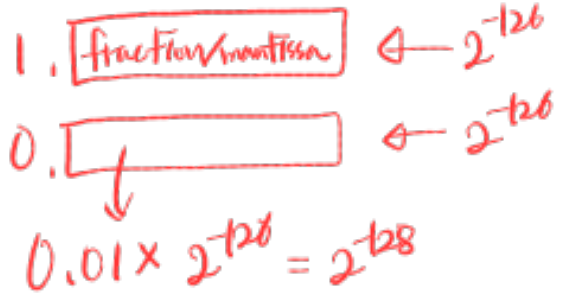
- Exponent representation = 11111111, Fraction = 00000...00
  - ±Infinity
  - Can be used in subsequent calculations, avoiding need for overflow check
- Exponent = 11111111, Fraction ≠ 00000...00
  - Not-a-Number (NaN)
  - Indicates illegal or undefined result
    - e.g., 0.0 / 0.0



Denormalized Numbers

- What would be the smallest number without the denormalized numbers?
  - Suppose we can use “00000000” as a normal exponent: $1.0\times2^{-127} = 5.87\ldots\times10^{-39}$
  - May not be small enough for some cases..
    - e.g., When $0 < |x - y| < 2^{-127}$, calculating $\frac{1}{x-y}$ would result in a divide by zero.

- How can we represent numbers smaller than 2^{-127} ?
  - First, reserve the exponent “00000000” as a special case
    - Now, the smallest number under the normalized form is 2^{-126}
  - For numbers smaller than 1.0×2^{-126}, use the “denormalized number” format.



Denormalized Numbers

- Exponent = 000...0

$$x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-(\text{Bias}-1)}$$

$\pm 0.\square \times 2^{-126}$ (127 → −126)

- Denormalized numbers do not assume the significand starts with “1.0”.
  - The integer part of the significand is “0.”
  - The magnitude is always $\times 2^{-126}$

- Denormalized number example: $1.01_2 \times 2^{-130} = 0.000101_2 \times 2^{-126}$
  - Exponent = 00000000
  - Mantissa = $00010100\ldots00_2$

23 bits: 000 101 … 000 (20 bits 이하 생략) → precision이 더 limited됨

- The effective precision is decreased compared to the normalized numbers. precision↓

New FP Formats